
UNIT 2 RELATIONAL DATABASE

Structure

Page Nos.

- 2.0 Introduction
- 2.1 Objectives
- 2.2 The Relational Model
 - 2.2.1 Domain, Attribute, Tuple and Relation
 - 2.2.2 Super keys Candidate keys and Primary keys for the Relations
- 2.3 Relational Constraints
 - 2.3.1 Domain Constraint
 - 2.3.2 Key Constraint
 - 2.3.3 Integrity Constraint
 - 2.3.4 Update Operations and Dealing with Constraint Violations
- 2.4 Relational Algebra
 - 2.4.1 Basic Set Operation
 - 2.4.2 Cartesian Product
 - 2.4.3 Relational Operations
- 2.5 Summary
- 2.6 Solution/Answers

2.0 INTRODUCTION

In the first unit of this block, you have been provided with the details of the Database Management System, its advantages, structure, etc. This unit is an attempt to provide you information about relational model. The relational model is a widely used model for DBMS implementation. Most of the commercial DBMS products available in the industry are relational at the core.

The relational model is based on the theory of relations and was first proposed by E.F. Codd. In this unit we will discuss the terminology, operators and operations used in relational model.

2.1 OBJECTIVES

After going through this unit, you should be able to:

- describe relational model and its advantages;
- perform basic operations using relational algebra;
- identify the relational constraints in a database system;
- identify the key of a relation.

2.2 THE RELATIONAL MODEL

A model of a database system basically defines the *structure or organisation of data*, the *set of integrity constraints* on the data and a *set of operations* that can be performed on the data. The basic structure of data in a relational model is in the form of two-dimensional tables. A table in this model is called a relation. Such organisation of data simplifies enforcement of integrity constraints and database operations. The following table represents a simple relation:

PERSON_ID	NAME	AGE	ADDRESS
1	Sanjay Prasad	35	B-4,Modi Nagar, UP
2	Sharada Gupta	30	Pocket 2, Mayur Vihar, Delhi
3	Vibhu Datt	36	C-2, Saket, New Delhi

Figure 1: A Sample Person Relation

Following are some of the advantages of the relational model:

- **Ease of use:** The simple tabular representation of the database helps the user define and query the database conveniently. For example, you can easily find out the age of the person whose first name is “Vibhu”.
- **Flexibility:** Since the database is a collection of tables, new data can be added and deleted easily. Also, manipulation of data from various tables can be done easily using various basic operations. For example, you can add a telephone number field in the table in *Figure 1*.
- **Accuracy:** In relational databases the relational algebraic operations are used to manipulate data values in a database. These are mathematical operations and ensure accuracy (and less of ambiguity) as compared to other models. These operations are discussed in more detail in Section 2.4.

2.2.1 Tuple, Attribute, Domain and Relation

Before we discuss the relational model in more detail, let us first define some very basic terms used in this model.

Tuple: Each row in a table represents a record or information of a single object, which is also termed as a tuple. For example, Figure 1 has three records or tuples. A record/tuple consists of a number of attributes, which is defined next.

Attribute: A relation consists of a large number of columns. Each of these columns, which defines a separate data value, is termed as an attribute. The column name in a relation is generally related to the meaning of data items of that column. For example, *Figure 2* represents a relation PERSON. The columns PERSON_ID, NAME, AGE, ADDRESS and TELEPHONE are the attributes of the relation PERSON and each row in the relation represents a separate tuple (record).

Relation Name: PERSON

PERSON_ID	NAME	AGE	ADDRESS	TELEPHONE
1	Sanjay Prasad	35	B-4,Modi Nagar, UP	011-25347527
2	Sharada Gupta	30	Pocket 2, Mayur Vihar, Delhi	023-12245678
3	Vibhu Datt	36	C-2, Saket, New Delhi	033-1601138

Figure 2: An extended PERSON relation

The relation of Figure 2 consists of 5 attributes, therefore, each tuple in this relation is called a 5-tuple. Thus, if a relation has n attributes, then each record in that relation would be termed as n -tuple.

Domain: A domain is a set of permissible values that can be accepted by a specific attribute. For example, in Figure 2, if you assume that there may be a maximum of 100 persons in the relation, then you may assign PERSON_ID to a domain of integer values, which should be in the range from 1 to 100 only. Once you have assigned this domain, then you will not be able to assign any value below 1 and above 100 to PERSON_ID. The domain of attribute AGE can be integer values between 0 and 150. The domain can be defined by assigning a type or a format or a range to an attribute. For example, a domain for a number 501 to 999 can be specified by having a 3-digit number format having a range of values between 501 and 999. Domains need not be contiguous numbers. For example, the enrolment number of IGNOU has the last digit as the check digit, thus the enrolment numbers are non-continuous.

Relation: Each table is a relation. A relation is defined using two basic aspects, viz. schema and an instance.

Relational Schema: A relational schema denoted as R , specifies the relation's name, the list of attributes of the relation and the domain of each attribute, which is denoted as $R(A_1, A_2, \dots, A_n)$. The relation R has n attributes, which is also called the degree of a relation.

For example, the relation schema of the relation PERSON (see Figure 1) can be defined as the follows:

PERSON (PERSON_ID: Integer, NAME: Character, AGE: Integer, ADDRESS: Character)

Since the PERSON relation contains four attributes, this relation is of degree 4.

Relation Instance or Relation State: A relation instance denoted as r is a collection of tuples for a given relational schema at a specific point of time.

A relation state r of the relation schema $R(A_1, A_2, \dots, A_n)$, also denoted by $r(R)$ is a set of n -tuples

$r = \{t_1, t_2, \dots, t_m\}$ has m tuples
Where each n -tuple is an ordered list of n values
 $t = \langle v_1, v_2, \dots, v_n \rangle$
where each v_i belongs to domain (A_i) or contains null values.

Let us elaborate the definitions above with the help of examples:

Example 1:

RELATION SCHEMA For STUDENT:

STUDENT (RollNo: string, name: string, login: string, age: integer)

RELATION INSTANCE

STUDENT				
	ROLLNO	NAME	LOGIN	AGE
t_1	3467	Shikha	xyz@hotmail.com	21
t_2	4677	Kanu	abc@gmail.com	20

Where $t_1 = (3467, shikha, xyz@hotmail.com, 20)$ for this relation instance, the number of tuples $m = 2$ and each tuple contains $n = 4$ values.

Example 2:

RELATIONAL SCHEMA For PERSON:

PERSON (PERSON_ID: integer, NAME: string, AGE: integer, ADDRESS: string)

RELATION INSTANCE

In this instance, the number of tuples $m = 3$ and each tuple has $n = 4$ values.

PERSON_ID	NAME	AGE	ADDRESS
1	Sanjay Prasad	35	B-4, Modi Nagar, UP
2	Sharad Gupta	30	Pocket 2, Mayur Vihar, Delhi
3	Vibhu Datt	36	C-2, Saket, New Delhi

If a relation has proper integrity constraints, then each tuple of that relation would be valid tuple. It may be noted that 'Null' value can be assigned for some of the attributes where the values are unknown or missing. However, the Null' value cannot be assigned to certain attributes, this will be explained in the later sections.

Ordering of tuples

In a relation, tuples are not inserted in any specific order. **Ordering of tuples is not defined as a part of a relation definition.** However, records may be organised later according to some attribute value in the storage system. For example, records in the PERSON table may be organised according to PERSON_ID. Such data organisation depends on the requirement of the underlying database application. However, for the purpose of display, you may get these tuples displayed in the sorted order of age. The following table is sorted by age. Please note that this is sorted relation is the same as that or relation displayed in the order of PERSON_ID as each and every tuple is the same. It is also worth mentioning that the relational model **does not allow duplicate tuples.**

PERSON

PERSON_ID	NAME	AGE	ADDRESS
2	Sharad Gupta	30	Pocket 2, Mayur Vihar, Delhi
1	Sanjay Prasad	35	B-4, Modi Nagar, UP
3	Vibhu Datt	36	C-2, Saket, New Delhi

2.2.2 Super Keys, Candidate Keys and Primary Keys for the Relations

As discussed in the previous section, ordering of relations does not matter and all tuples in a relation are unique. However, can you uniquely identify a tuple in a relation? To answer this question, let us discuss the concept of keys in relations.

Super Keys

A **super key** is an attribute or set of attributes used to identify the records uniquely in a relation.

For Example, in the Relation PERSON described earlier PERSON_ID is a super key since PERSON_ID is unique for each tuple/record. Similarly (PERSON_ID, AGE) and (PERSON_ID, NAME) are also super keys of the relation PERSON since their combination is also unique for each tuple/record.

Candidate keys

Super keys of a relation may contain extra attributes. A candidate key is a minimal super key, which means that a candidate key does not contain any extraneous attribute. An attribute is called extraneous if even after removing it from the key, the remaining attributes still has the properties of a super key. A relation may have several candidate keys. A candidate key may contain one or many attributes of a relation.

The following properties must be satisfied by a candidate key:

- In any instance of a relation, the value of the candidate key attribute(s) should be unique.
- You cannot put a 'Null' value in any attribute that is a part of the candidate key. This rule is also termed the entity integrity rule. Thus, a candidate key is unique and not null.
- A candidate key should have a minimal set of attributes.
- The value of a candidate key must be **stable**, which means it should not change frequently or its **value** change should not be outside the control of the system.

A relation can have more than one candidate key and one of them can be chosen as a **primary key**.

For example, in the relation PERSON, the two possible candidate keys are PERSON_ID and NAME (assuming unique names exist in the table). PERSON_ID may be chosen as the primary key.

Check Your Progress 1

Answer the following questions for the relational instance s of the following Supplier relation S:

S		
<u>SNO</u>	SNAME	CITY
S1	Smita	Delhi
S2	Jim	Pune

1. What are the different attributes of relation S and how many tuples does S have?
.....
2. What are the domains of the attributes of S?
.....
.....
3. On sorting this relation on the field CITY, will the relation change? Will the order of tuples in the relation change?
.....
4. List the super keys, all the possible candidate keys, and the primary key of the relation.
.....

2.3 RELATIONAL CONSTRAINTS

A relational database is a collection of relations. Each relation consists of tuples, which consist of attributes. You can associate constraints, called relational constraints, with the attributes. Such constraints can be of the following types:

- Domain Constraints
- Key Constraint
- Integrity Constraints

2.3.1 Domain Constraint

These constraints bind the possible values of the attribute in a relation to a specific set of values, called the domain of the attribute. For example, an attribute COUNTRY may have a domain of names of all the possible names of the countries. In commercial relational database management systems, the data type associated with an attribute defines the broad domain of an attribute. These domains include:

- 1) Integral Data type like integer, long etc.
- 2) Data types related to real numbers like float, double etc.
- 3) Data types relating to alphabets like characters, strings etc.
- 4) Boolean

Thus, domain constraint specifies the possible set of values that you want to put in an attribute of a relation. The values that appear in each attribute/column must be drawn from the domain associated with that column.

For example, consider the relation:

STUDENT

ROLLNO	NAME	LOGIN	AGE
3467	Shikha	xyz@hotmail.com	21
4677	Kanu	abc@gmail.com	20

In the relation above, AGE of the relation STUDENT always belongs to the integer domain within a specified range (say 0 representing just born to 150) and not to strings or any other domain. Within a domain, non-atomic values should be avoided. This sometimes cannot be checked by domain constraints. For example, a database which has area code and phone numbers as two different fields will take phone numbers as-

Area_code	Phone
11	29534466

A non-atomic value in this case for a phone can be 1129534466, however, this value can be accepted by the Phone field only.

2.3.2 Key Constraint

This constraint states that the key attribute value in each tuple must be unique, i.e., no two tuples can contain the same value for the key attribute. This is because the value of the primary key is used to identify a unique tuple in a relation.

Example 3: If A is the key attribute in the following relation R, then A1, A2 and A3 must be unique.

R	
A	B
A1	B1
A3	B2
A2	B2

Example 4: In relation PERSON, PERSON_ID is the primary key so PERSON_ID cannot be assigned the same for two different persons.

2.3.3 Integrity Constraint

There are two types of integrity constraints:

- Entity Integrity Constraint
- Referential Integrity Constraint

Entity Integrity Constraint:

It states that any attribute of a **primary key cannot take a Null value**. This is because the primary key is used to identify individual tuple in the relation. You will not be able to identify the records uniquely if they contain null values for the primary key attributes. This constraint is specified for each relation of a database.

Example 5: Let R be a relation; an instance *r* of R is given below. Is the instance *r* valid?

A#	B	C
Null	B1	C1
A1	B2	C2
Null	B2	C3
A2	B3	C3
Null	B1	C5

Note: A '#' in the headings row is indicating that A is the Primary key of R.

In the instance r of relation R above, the primary key has Null values in the tuple t_1 , tuple t_3 and tuple t_5 . As per the entity integrity constraint, Null value in primary key is not permitted. Thus, relation instance r is an invalid instance.

Referential integrity constraint

For defining the referential integrity constraint, first we explain the concept of a **foreign key** and **foreign key constraint**.

Consider an attribute set A of a relation R, which references an attribute set B in a relation S, then A will be referred to foreign key in R provided it fulfills the following conditions:

1. B is the Primary key of S.
2. A and B are defined over the same domains. A is called the foreign key in relation R, which references B in relation S.
3. For every value x of attribute set A, in any instance r of R, there exists a tuple in any instance s of S, where the value of attribute set B is x . Please note that there will be only one such tuple having the value x , as B is the Primary key of S. This is called foreign key constraint.
4. Please note that the relation R is a referencing relation and S is called referenced relation.

A referential integrity constraint is a more generic form of foreign key constraint, which requires that the attribute of the referencing relation must be present in the attribute of referenced relation in some tuple. Thus, foreign key constraint is a specific form of referential integrity constraint. Foreign key constraint is explained with the help of the following example.

Example 6: Consider the two relational instance r of relation R and s of relation S.

Now answer the following questions:

- a) If C in R is foreign key of C in S, then are the following instances r and s valid?
- b) A new tuple having values (A6, B2, C4) is added in r . Does it violate foreign key constraint?

Instance r of R

A#	B	C^
A1	B1	C1
A2	B2	C2
A3	B3	C3
A4	B4	C3
A5	B1	C5

Instance s of S

E	C#
E1	C1
E2	C3
E3	C5
E2	C2

Notes:

- '#' identifies the Primary key of a relation.
- '^' identifies the foreign key in a relation.

- a) In the example above, every value of C in r is matching with the values of C in s in some tuple. The r contains the values C1, C2, C3, C5 and s contains all these values. Thus, the instances r and s does not violate foreign key constraint.
- b) If you try to insert the tuple (A6, B2, C4) in r, then it is violating the foreign key constraint for the present instances of relations, as there is no tuple in s, which contains value C4 in s.

2.3.4 Data Updating Operations and Constraint Violations

There are three basic data updating operations to be performed on relations:

- Insertion of tuples
- Deletion of tuples
- Update the value of attribute(s) in a relation

The INSERT Operation:

- To add new records in a database system, you use insert operation. For example, in the instance r of R of example 6, you may like to add a new record <'A5', 25, 'C6'>. Addition of a new record may cause constraint violation, which are explained below:
- Violation of Domain constraint: Such a violation occurs if the domain of a value, which you are trying to insert in an attribute, does not match the attribute's domain. e inserting a numeric value 25 into attribute B, whose domain is alphanumeric sting. This will cause a domain constraint violation.
- Violation of Key constraint: This violation occurs when you try to insert a key value, which already exists in some tuple of existing relational instance. For example, when you are trying to insert tuple <'A5', 25, 'C6'> in R, you are inserting a value A5 in the key attribute A. However, the value A5 already exists in attribute A of tuple t₅. Therefore, it is a violation of key constraint.
- Violation of Entity Integrity constraint: This violation will occur; in case you try to insert a Null value into the primary key attribute. For example, if you try to insert a tuple <'Null, 25, 'C6'> in R, you are violating this constraint.
- Violation of Referential Integrity constraint: Consider that while inserting a new tuple, the value that you are trying to insert in a foreign key attribute, which refers to an attribute in another relation where it is the primary key, does not match with any value of referred attribute of the referenced relation. (Please see example 6 par (b)). For example, when you are trying to insert tuple <'A5', 25, 'C6'> in R, you are inserting a value C6 in the foreign key attribute A of R. However, the value C6 is not a value in referred attribute C in referenced relation S. Thus, insertion of this tuple violates the referential integrity constraint.

Dealing with constraints violation during insertion:

If the *insertion* violates one or more constraints, then two options are available:

- Default option: - *Insertion can be rejected and the reason for rejection can also be explained to the user by DBMS.*
- Ask the user to correct the data, resubmit, and give *the reason for rejecting the insertion.*

Example 7:

Consider the Relation PERSON of Example 2:

PERSON

PERSON_ID	NAME	AGE	ADDRESS
1	Sanjay Prasad	35	B-4,Modi Nagar, UP
2	Sharad Gupta	30	Pocket 2, Mayur Vihar, Delhi
3	Vibhu Datt	36	C-2, Saket, New Delhi

(1) Insert <1, 'Vipin', 20, 'Mayur Vihar'> into PERSON

Violated constraint: - Key constraint

Reason: - Primary key 1 already exists in PERSON.

How to resolve: - DBMS could ask the user to provide valid PERSON_ID value and accept the insertion if valid PERSON_ID value is provided.

(2) Insert <'null', 'Anurag', 25, 'Patparganj'> into PERSON

Violated constraint: - Entity Integrity constraint

Reason: - Primary key is 'null'.

How to resolve: - DBMS could ask the user to provide valid PERSON_ID value and accept the insertion if valid PERSON_ID value is provided.

(3) Insert <'abc', 'Suman', 25, 'IP college'> into PERSON

Violated constraint: - Domain constraint

Reason: - value of PERSON_ID is given a string which is not valid.

How to resolve: - DBMS could ask the user to provide valid PERSON_ID value and accept the insertion if valid PERSON_ID value is provided.

(4) Insert <10, 'Anu', 25, 'Patparganj'> into PERSON

Violated constraint: - None

Note: In the first 3 cases of constraint violations above DBMS will reject the insertion.

The Deletion Operation:

The delete operation is used to delete some existing records from a relation. To delete some specific records from the database a condition is specified based on which records are selected for deletion.

Constraints that can be violated during deletion

Only one type of constraints can be violated during deletion, it is referential integrity constraint. When you want to delete a record in a referenced relation, there is a possibility that you may delete a tuple whose attribute is being referenced by the referencing relation, where the attribute is the foreign key. Please go through example 8 very carefully.

Dealing with Constraints Violation

If the *deletion* violates the referential integrity constraint, then three options are available:

- Default option: - *Reject the deletion*. It is the job of the DBMS to explain to the user why the deletion was rejected.
- *Attempt to cascade (or propagate) the deletion* by deleting tuples whose foreign key references the tuple that is being deleted.
- Change the value of the *referencing attribute* that causes the violation.

Example 8:

Let instance r of relation R be:

A#	B	C^
A1	B1	C1
A2	B3	C3
A3	B4	C3
A4	B1	C5

And instance q of relation Q be:

C#	D
C1	D1
C3	D2
C5	D3

Note:

- 1) '#' identifies the Primary key of a relation.
 - 2) '^' identifies the Foreign key of a relation.
- (1) Delete a tuple with C# = 'C1' in Q.
Violated constraint: - Referential Integrity constraint is violated
Reason: - If the stated tuple is deleted from Q, the first tuple of R, which contains the Foreign key (C) value C1, will violate the foreign key constraint.
How to resolve: Options available are
- 1) Reject the deletion.
 - 2) DBMS may automatically delete all tuples from relation Q and S with C # = 'C1'. This is called cascade deletion.
 - 3) The third option would result in putting NULL value in R where the value of attribute C is C1, which is the first tuple of R.

The Update Operations:

Update operations are used for modifying the values of attributes in a database. The constraint violations faced by this operation are logically the same as the problem faced by Insertion-Deletion Operation. You may define these actions yourself.

Check Your Progress 2

- 1 Consider the tables Suppliers, Parts, proJect and SPJ relation instances in the relations below.

S

<u>SNO</u>	SNAME	CITY
S1	Smita	Delhi
S2	Jim	Pune
S3	Ballav	Pune

P

<u>PNO</u>	PNAME	COLOUR	CITY
P1	Nut	Red	Delhi
P2	Bolt	Blue	Pune
P3	Pen	White	Mumbai

J

<u>JNO</u>	JNAME	CITY
J1	Sorter	Pune
J2	Display	Mumbai

SPJ

<u>SNO</u>	<u>PNO</u>	<u>JNO</u>	QUANTITY
S1	P1	J1	200
S2	P2	J2	700

Using the instance of relations as given above, answer the following questions:

- 1 List a domain constraint for S.
.....
.....
- 2 List the Primary keys of all the relations and primary key constraint for the SPJ relation.
.....
.....
- 3 List all the Foreign keys and Foreign key constraints.
.....
.....
- 4 What would be the constraint violations if the following operations are performed:
 - (a) Insert <Null, 'Jack', 'Mumbai'> into S
 - (b) Insert <'S2', Null, 'J3', 200> into SPJ
 - (c) Insert <'P2', 'Pencil', 'Grey', 'Kolkata'> into P
 - (d) Insert <'J3', 'Monitor', 'Jaipur'> into J.....
.....
.....

2.4 RELATIONAL ALGEBRA

Relational Algebra is a set of basic operations used to manipulate the data in relational model. These operations enable the user to specify basic retrieval requests. The result of retrieval is a new temporary relation, which is computed after performing these operations on one or two relations. Some of these **operations can be classified in three categories:**

- Relational Operations
These can be unary operations
 1. SELECTION
 2. PROJECTIONOr binary operations
 1. CARTESIAN PRODUCT
 2. JOIN
- Basic Set Operations
 1. UNION
 2. INTERSECTION
 3. SET DIFFERENCE
- Assignment Operation, rename and other operations

In this section, we will discuss relational operations and basic set operations. You may refer to the other operations from the further readings.

2.4.1 Relational Operations

Relational algebra forms the basis of the query languages on a database system. Thus, by studying relational algebra, you may be able to write better queries for a database system. Let us discuss these operations in detail.

Selection Operation:

Selection is a unary operator, that is used to choose only those tuples from a relation that fulfill a given criteria. It is represented using the mathematical symbol σ , as given below:

σ <Selection condition> (Relation)

You should specify a Boolean expression as a <Selection condition>. A selection condition uses comparison operators like $\{ \leq, \geq, \neq, =, <, > \}$ and logical connectors like and ($\wedge$), or ($\vee$) and not ($\neg$).

Example 13:

Consider the relation PERSON. If you want to display details of persons having age less than or equal to 30, then the selection operation will be used as follows:

$\sigma_{AGE \leq 30}(\text{PERSON})$

The resultant relation will be as follows:

PERSON_ID	NAME	AGE	ADDRESS
2	Sharad Gupta	30	Pocket 2, Mayur Vihar, Delhi

Note:

- 1) Selection operation is commutative, i.e.,

$$\sigma_{\langle \text{condition1} \rangle}(\sigma_{\langle \text{condition2} \rangle}(\text{R})) = \sigma_{\langle \text{condition2} \rangle}(\sigma_{\langle \text{condition1} \rangle}(\text{R}))$$
Hence, you can apply a sequence of Selection operations in any order
- 2) You can apply more than one condition by using logical connectors.

Projection operation

The projection operation is used to select specific attributes from amongst all the attributes of records. This is denoted as Π .

$\Pi_{\text{List of attributes for projection}}(\text{Relation})$

Example 14:

Consider the relation PERSON. If you want to display only the names of persons, then the projection operation will be used as follows:

$\Pi_{\text{NAME}}(\text{PERSON})$

The resultant relation will be as follows:

NAME
Sanjay Prasad
Sharad Gupta
Vibhu Datt

Please note that for projection operation:

$\Pi_{\langle \text{List1} \rangle}(\Pi_{\langle \text{list2} \rangle}(\text{R})) = \Pi_{\langle \text{list1} \rangle}(\text{R})$ provided $\langle \text{list2} \rangle$ contains attributes in $\langle \text{list1} \rangle$.

2.4.2 Cartesian Product

Given two relations R and S, the cartesian product of these two relations can be represented as $T = R \times S$. The cartesian product operation produces all possible combinations of the tuples of relations R and S. The cartesian product operation is commutative as well as associative. In addition, the degree of the resultant

relation (T) is the sum of the degrees of relation R and relation S. The Cartesian product can be expressed using the following mathematical expression:

$T = \{t_1 \parallel t_2 \mid t_1 \in R_1 \wedge t_2 \in R_2\}$. Here, $\parallel$ is a concatenation operator.

Example 12: Consider the following two relational instances of relation R1 and R2. What would be the cartesian product of the relations?

R1

A	B
A1	B1
A1	B2
A2	B2
A2	B3

R2

C
C1
C2

The Cartesian produce of the relations is denoted by $R_3 = R_1 \times R_2$. The relation R3 will be as shown below:

A	B	C
A1	B1	C1
A1	B1	C2
A1	B2	C1
A1	B2	C2
A2	B2	C1
A2	B2	C2
A2	B3	C1
A2	B3	C2

Please note that in cartesian product every tuple/record of R1 is concatenated with every record of R2. You can observe that the record $\langle A1, B1 \rangle$ is concatenated with both the records of R2, that is why in the output R3 you have tuples $\langle A1, B1, C1 \rangle$ and $\langle A1, B1, C2 \rangle$, likewise for all the tuples of R1. Please also note that R1 has 4 tuples and R2 has 2 tuples, therefore, R3 has $4 \times 2 = 8$ tuples.

The JOIN operation

JOIN is a binary operator. It combines two relations based on a given join condition.

A JOIN operation is represented by mathematical symbol $\bowtie$. But how does JOIN operation combine two relations? It would require that there should be at least one attribute in each of the relations, which have the same domain. Such attributes are called domain compatible attributes.

Syntax:

$R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$ is used to combine related tuples from two relations R1 and R2 into a single tuple.

$\langle \text{join condition} \rangle$ is of the form:

$\langle \text{condition} \rangle \text{AND} \langle \text{condition} \rangle \text{AND} \dots \text{AND} \langle \text{condition} \rangle$.

- Degree of Relation:

$\text{Degree}(R_1 \bowtie_{\langle \text{join condition} \rangle} R_2) \leq \text{Degree}(R_1) + \text{Degree}(R_2)$.

- Three types of joins are there:

a) Theta (θ) join

When each condition is of the form $A \theta B$, where A is an attribute of R1 and B is an attribute of R2; both A and B have the same domain; and θ is one of the comparison operators $\{\leq, \geq, \neq, =, <, >\}$.

b) **Equijoin**

Equijoin is a restricted form of When each condition appears with equality operator (=) only.

c) **Natural join**

Natural join is defined as a specialised type of join, when the joining attributes in the two joining relations have the identical name, in addition to the identical domain and the condition for the join operation is equality (=) condition. In such cases only one joining attribute is kept in the result.

The following example explains the Natural join operation.

Example 15:

Consider the instance of the following relations. The primary key of STUDENT relation is ROLLNO and foreign key is COURSE_ID, which references the primary key COURSE_ID of COURSE relation.

STUDENT

ROLLNO	NAME	ADDRESS	COURSE_ID
100	Kanupriya	234, Saraswati Vihar.	CS1
101	Rajni Bala	120, Vasant Kunj	CS2
102	Arpita Gupta	75, SRM Apartments.	CS4

COURSE

COURSE_ID	COURSE_NAME	DURATION
CS1	MCA	3yrs
CS2	BCA	3yrs
CS3	M.Sc.	2yrs
CS4	B.Sc.	3yrs
CS5	MBA	2yrs

Display the name and other details of all the students along with their course details.

Solution: You may observe that the course details are existing in the COURSE relation and student names and other details are in the STUDENT relation. Therefore, you need to join these two relations to extract information. The join attributes in this case are COURSE_ID in STUDENT and COURSE_ID in the COURSE relation and equality operator is to be used; therefore, you use natural join operation, which can be represented as:

$$\text{STUDENTCOURSE} = \text{STUDENT} \bowtie \text{COURSE}$$

The output of this natural join will be the following relation. Please note that the output has just one COURSE_ID attribute.

STUDENTCOURSE

ROLLNO	NAME	ADDRESS	COURSE_ID	COURSE_NAME	DURATION
100	Kanupriya	234, Saraswati Vihar.	CS1	MCA	3yrs
101	Rajni Bala	120, Vasant Kunj	CS2	BCA	3yrs
102	Arpita Gupta	75, SRM Apartments.	CS4	B.Sc.	3yrs

There are other types of joins like outer joins. You must refer to further reading for more details on those operations. They are also explained in later blocks of this course.

2.4.3 Basic Set Operation

The set operations are the binary operations, i.e., each is applied to two sets or relations, which should be union compatible. Two relations $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_n)$ are said to be union compatible if they have the **same degree n** and domains of the corresponding attributes are also the same, i.e., **Domain $(A_i) = \text{Domain}(B_i)$ for $1 \leq i \leq n$** .

Union

If R_1 and R_2 are two union compatible relations, then $R_3 = R_1 \cup R_2$ is the relation containing tuples that are either in R_1 or in R_2 or in both. In other words, R_3 will have tuples such that $R_3 = \{t \mid t \in R_1 \vee t \in R_2\}$.

Example 9:

R1

A	B
A1	B1
A2	B2
A3	B3
A4	B4

R2

X	Y
A1	B1
A7	B7
A2	B2
A4	B4

$R_3 = R_1 \cup R_2$ is

R3

A	B
A1	B1
A2	B2
A3	B3
A4	B4
A7	B7

Note: 1) Union is a commutative operation, i.e.,

$$R \cup S = S \cup R.$$

2) Union is an associative operation, i.e.,

$$R \cup (S \cup T) = (R \cup S) \cup T.$$

Intersection

If R_1 and R_2 are two union compatible functions or relations, then the result of $R_3 = R_1 \cap R_2$ is the relation that includes all tuples that are in both the relations

In other words, R_3 will have tuples such that $R_3 = \{t \mid t \in R_1 \wedge t \in R_2\}$.

Example 10:

R1

A	B
A1	B1
A2	B2
A3	B3
A4	B4

R2

X	Y
A1	B1
A7	B7
A2	B2
A4	B4

$R3 = R1 \cap R2$ is

A	B
A1	B1
A2	B2
A4	B4

Note: 1) Intersection is a commutative operation, i.e.,

$$R1 \cap R2 = R2 \cap R1.$$

2) Intersection is an associative operation, i.e.,

$$R1 \cap (R2 \cap R3) = (R1 \cap R2) \cap R3$$

Set Difference

If $R1$ and $R2$ are two union compatible relations, then the result of $R3 = R1 - R2$ is the relation that includes only those tuples that are in $R1$ but not in $R2$. In other words, $R3$ will have tuples such that $R3 = \{t \mid t \in R1 \wedge t \notin R2\}$.

Example 11:

$R1$

X	Y
A1	B1
A7	B7
A2	B2
A4	B4

$R2$

A	B
A1	B1
A2	B2
A3	B3
A4	B4

$R1 - R2 =$

A	B
A7	B7

$R2 - R1 =$

A	B
A3	B3

Note: -1) Difference operation is not commutative, i.e.,

$$R1 - R2 \neq R2 - R1$$

2) Difference operation is not associative, i. e.,

$$R1 - (R2 - R3) \neq (R1 - R2) - R3$$

Please note that a relational query language would be relationally complete, if it supports five relational algebraic operators – Selection, Projection, Union, Set Difference and Cartesian Product.

Check Your Progress 1

- 1) A database system is fully relational if it supports a language as powerful as _____.
- 2) Primitive operations are union, difference, product, selection and projection. The $A \cap B$ can be computed using
- 3) Which of the following is not a traditional set operator in relational algebra?

- Union
- Intersection
- Difference
- Join

- 4) Consider the relational instances of the relations Suppliers, Parts, proJect and SPJ relations given below. (Underline represents a key attribute. The SPJ relation has three Foreign keys: SNO, PNO and JNO.)

S			P			
<u>SNO</u>	SNAME	CITY	<u>PNO</u>	PNAME	COLOUR	CITY
S1	Smita	Delhi	P1	Nut	Red	Delhi
S2	Jim	Pune	P2	Bolt	Blue	Pune
S3	Ballav	Pune	P3	Part1	White	Mumbai
S4	Seema	Delhi	P4	Part2	Blue	Delhi
S5	Salim	Agra	P5	Camera	Brown	Pune
			P6	Part3	Grey	Delhi

J			SPJ			
<u>JNO</u>	JNAME	CITY	<u>SNO</u>	<u>PNO</u>	<u>JNO</u>	QUANTITY
J1	Sorter	Pune	S1	P1	J1	200
J2	Display	Bombay	S1	P1	J4	700
J3	OCR	Agra	S2	P3	J2	400
J4	Console	Agra	S2	P2	J7	200
J5	RAID	Delhi	S2	P3	J3	500
J6	EDP	Udaipur	S3	P3	J5	400
J7	Tape	Delhi	S3	P4	J3	500
			S3	P5	J3	600
			S3	P6	J4	800
			S4	P6	J2	900
			S4	P6	J1	100
			S4	P5	J7	200
			S5	P5	J5	300
			S5	P4	J6	400

Using the in the relations above, which of the following operations and constraints would be valid:

- UPDATE Project J7 in J setting CITY to Nagpur.
- UPDATE part P5 in P, setting PNO to P4.
- UPDATE supplier S5 in S, setting SNO to S8, if the relevant update rule is RESTRICT.
- DELETE supplier S3 in S, if the relevant rule is CASCADE.
- DELETE part P2 in P, if the relevant delete rule is RESTRICT.
- DELETE project J4 in J, if the relevant delete rule is CASCADE.
- UPDATE shipment S1-P1-J1 in SPJ, setting SNO to S2. (shipment S1-P1-J1 means that in SPJ table the value for attributes SNO, PNO and JNO are S1, P1 and J1 respectively)
- UPDATE shipment S5-P5-J5 in SPJ, setting JNO to J7.
- UPDATE shipment S5-P5-J5 in SPJ, setting JNO to J8
- INSERT shipment S5-P6-J7 in SPJ.
- INSERT shipment S4-P7-J6 in SPJ
- INSERT shipment S1-P2-jjj (where jjj stands for a default project number).

.....

.....

.....

- 5) Find the name of projects in the relations above, to which supplier S1 has supplied using relational algebra.

.....
.....
.....
.....

2.5 SUMMARY

This unit is an attempt to provide a detailed viewpoint of database design. The topics covered in this unit include the relational model including the representation of relations, operations such as set type operators and relational operators on relations. The E-R model explained in this unit covers the basic aspects of E-R modeling. E-R modeling is quite common to database design, therefore, you must attempt as many problems as possible from the further reading. The E-R diagram has also been extended. However, that is beyond the scope of this unit. You may refer to further readings for more details on E-R diagrams.

2.6 SOLUTIONS/ ANSWERS

Check Your Progress 1

1. Supplier Number (SNO), Supplier Name (SNAME) and City of the location of the supplier (CITY). The present instance s of the relation S has 2 tuples.
2. Domain of SNO is the codes that are being assigned to suppliers. The present coding uses the first character as S followed by the sequence number of a supplier. The domain of SNAME is strings of characters of some defined length and the domain of CITY is the set of possible cities in India.
3. The relation will not change; the order of tuples will change.
4. The super key of the relation could be (SNO, SNAME, CITY) or (SNO, SNAME) or (SNO, CITY) or assuming every supplier has unique name (SNAME, CITY) or SNO or assuming every supplier has unique name SNAME.
The possible candidate keys are SNO or assuming every supplier has unique name SNAME
Primary key may be selected as SNO and alternate key would be SNAME

Check Your Progress 2

1. Domain constraint for S will be for CITY which should be checked to be in a selected list of cities of India. SNO may be checked by a Range constraint and SNAME may be data type with maximum allowable length of name.
2. The Primary keys of relation S, P and J are SNO, PNO and JNO respectively. The primary key of SPJ is a composite key (SNO, PNO, JNO). As per the primary key constraint none of the attributes of primary key in SPJ, viz. SNO, PNO or JNO can be null.
3. There are three foreign keys, all of them are in SPJ relation. These are (i) SNO (ii) PNO (iii) JNO. SNO of SPJ references attribute SNO in S; PNO of SPJ references attribute PNO in P; and JNO of SPJ references attribute JNO in J. As per foreign key constraints the values of SNO, PNO and JNO in SPJ can be only as per the related referenced attributes.
4. The violations are as under:
 - a. Primary key of S is SNO, it cannot be Null, you may change it to <S4, Jack, Mumbai>, which will be inserted successfully in S.

- b. Primary key violation in SPJ as PNO cannot be Null as it is a part of primary key. There is another violation here, foreign key JNO has a value J3, which does not exist in JNO attribute of relation J for this instance, thus, the allowable values for JNO in SPJ is just J1 or J2. Thus, a correct record insertion in SPJ would be: <'S2', 'P3', 'J2', 200>.
- c. The primary key 'P2' already exists and cannot be duplicated, the correct insertion may be to Insert <'P4, 'Pencil', 'Grey', 'Kolkata'> into P.
- d. There is no violation.

Check Your Progress 3

1. relational algebra
2. $A - (A - B)$
3. (d) Join
4.
 - i. Accepted
 - ii. Rejected (candidate key uniqueness violation)
 - iii. Rejected (violates RESTRICTED update rule, as SPJ contains tuples having value S5 in SNO)
 - iv. Accepted (supplier S3 and all shipments for supplier S3 in the relation SPJ would be deleted, as the rule is CASCADE)
 - v. Rejected (violates RESTRICTED delete rule, as SPJ contains tuples having value P2 in PNO)
 - vi. Accepted (project J4 and all shipments for project J4 from the relation SPJ are deleted)
 - vii. Accepted
 - viii. Rejected (primary/candidate key uniqueness violation as tuple S5-P5-J7 already exists in relation SPJ)
 - ix. Rejected (referential integrity violation as there exists no tuple for J8 in relation J)
 - x. Accepted
 - xi. Rejected (referential integrity violation as there exists no tuple for P7 in relation P)
 - xii. Rejected (referential integrity violation – the default project number jjj does not exist in relation J).
- 5) The answer to this query will require the use of the relational algebraic operations. This can be found by performing selection of supplies made by S1 in SPJ, then taking projection of resultant on JNO and joining the resultant to J relation. Let us show steps:

First find out the supplies made by supplier S1 by selecting those tuples from SPJ where SNO is S1. The relation operator being:

$$SPJT = \sigma_{<SNO = 'S1'>} (SPJ)$$

The resulting temporary relation SPJT will be:

<u>SNO</u>	<u>PNO</u>	<u>JNO</u>	<u>QUANTITY</u>
S1	P1	J1	200
S1	P1	J4	700

Next, you may take projection of SPJT on PNO

$$SPJT2 = \Pi_{JNO} (SPJT)$$

The resulting temporary relation SPJT will be:

<u>JNO</u>
J1
J4

Next, take natural JOIN this table with J:

$$\text{RESULT} = \text{SPJT2} \bowtie \text{J}$$

The resulting relation RESULT will be:

<u>JNO</u>	JNAME	CITY
J1	Sorter	Pune
J4	Console	Agra

You can write it as a single relational algebraic query as:

$$\text{RESULT} = (\Pi_{\text{JNO}} (\sigma_{\text{<SNO = 'S1'>}} (\text{SPJ}))) \bowtie \text{J}$$

